Software Requirements Specification

Version 1.0

Project Information

Project:	**Food Ordering System**
Document:	**SRS — Phase 1 Midterm**
Subject:	**Fundamentals of Software Engineering**
Version:	**1.0**
Date:	**March 23, 2026**
Stack:	**PHP · MySQL · HTML/CSS · MVC**

Contents

1 Introduction **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Definitions and Acronyms 1
1.4 Document Overview 1

2 Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Assumptions and Dependencies 3

3 Functional Requirements **3**

4 Non-Functional Requirements **6**

5 Detailed Class Descriptions **8**

6 Use Case Descriptions **10**

7 Database Schema **13**

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete set of functional and non-functional requirements for the **Food Ordering System** — a PHP-based web application developed as a Phase 1 midterm project for the Fundamentals of Software Engineering course.

The system allows customers to browse food menus, place orders via an interactive cart, and print invoices, while providing administrators with full CRUD control over all seven database entities.

1.2 Scope

The Food Ordering System covers the following functional scope:

- Role-based authentication with **Admin** and **Customer** roles.
- Full CRUD management for **7 database tables**: `users`, `user_types`, `menu`, `products`, `orders`, `order_details`, and `usertype_menu`.
- Customer-facing menu browsing with a live cart and one-click ordering.
- Printable, formatted invoices for every placed order.
- Admin dashboard with real-time statistics.
- Role-to-menu mapping management (`usertype_menu`).

1.3 Definitions and Acronyms

Term	Definition
CRUD	Create, Read, Update, Delete — the four fundamental database operations
MVC	Model-View-Controller — the architectural pattern used throughout the project
PDO	PHP Data Objects — the database abstraction layer for all SQL queries
SRS	Software Requirements Specification — this document
Admin	A privileged user who manages all system data
Customer	A regular end-user who browses menus and places food orders
Session	PHP server-side mechanism (`$_SESSION`) for maintaining login state
FR	Functional Requirement
NFR	Non-Functional Requirement
bcrypt	Password hashing algorithm used via PHP's `password_hash()`
LAMP	Linux, Apache, MySQL, PHP — the server stack the system targets

1.4 Document Overview

- **Section 2** — Overall system description, user classes, and constraints.

- **Section 3** — Functional requirements (15 requirements).
- **Section 4** — Non-functional requirements (6 requirements).
- **Section 5** — Detailed class descriptions (17 classes).
- **Section 6** — Use case descriptions (10 use cases).
- **Section 7** — Database schema overview.

2 Overall Description

2.1 Product Perspective

The Food Ordering System is a **standalone web application** built in pure PHP using a custom MVC architecture with no external framework dependency. It runs on any standard LAMP/WAMP/XAMPP stack and uses MySQL as its relational database backend.

The system replaces a manual, paper-based ordering process with a digital, role-aware interface that serves both kitchen staff (Admin) and walk-in or remote customers.

2.2 Product Functions

- **Authentication** — Login/logout with bcrypt-hashed passwords and PHP session management.
- **Admin Panel** — Dashboard with summary statistics, and full CRUD for all 7 database tables via modal dialogs.
- **Customer Interface** — Browse menu categories, add items to an interactive cart, and place orders in a single step.
- **Order Lifecycle** — Status tracking through *pending* → *processing* → *delivered* (or *cancelled*).
- **Invoice Printing** — Formatted, print-ready invoice for any order, accessible by both Admin and Customer.

2.3 User Classes and Characteristics

User Class	Description	Access
Admin	System administrator. Full control over all data and system configuration.	All pages: users, user types, menus, products, orders, order details, role-menu mapping, dashboards, invoices.
Customer	Registered end-user who places food orders.	Browse menu, place orders, view personal order history, print own invoices.

2.4 Operating Environment

- **Server:** Apache 2.4+ or Nginx with PHP 7.4 or higher.

- **Database:** MySQL 5.7+ or MariaDB 10.3+.
- **Client:** Any modern browser (Chrome, Firefox, Edge, Safari).
- **Development Stack:** XAMPP or WAMP (Windows), LAMP (Linux).

2.5 Assumptions and Dependencies

- The MySQL server is accessible at `localhost` with the credentials defined in `app/config/database.php`.
- The database `food_ordering_system` has been created and seeded by running `database.sql`.
- `public/setup.php` has been executed once to set correct bcrypt password hashes for demo accounts.
- No Composer or third-party PHP libraries are required.

3 Functional Requirements

Each requirement is identified by a unique ID, with a priority classification of **Must Have** (critical) or **Should Have** (important but not blocking).

ID	Name	Description	Priority	Impl. Reference
FR-01	User Authentication	The system shall allow users to log in using their email address and password. Invalid credentials redirect back to login with an error message.	Must Have	`LoginController::login()` with `password_verify()`
FR-02	Role-Based Dashboards	The system shall redirect authenticated users to role-specific dashboards: Admin to the admin panel, Customer to the customer home.	Must Have	`public/index.php` router; admin and customer dashboard views
FR-03	User Management (CRUD)	Admin shall create, read, update, and delete user accounts. Each user is assigned a role from the `user_types` table.	Must Have	`UserController`, `User` model, `users/index.php`
FR-04	User Type Management (CRUD)	Admin shall manage user roles with full CRUD on the `user_types` table.	Must Have	`UserTypeController`, `UserType` model
FR-05	Menu Management (CRUD)	Admin shall create, read, update, and delete menu categories. Each menu has a name and description.	Must Have	`MenuController`, `Menu` model

ID	Name	Description	Priority	Impl. Reference
FR-06	Product Management (CRUD)	Admin shall add, edit, and delete products. Each product has a name, price, and belongs to a menu category.	Must Have	`ProductController`, `Product` model
FR-07	Order Management (CRUD)	Admin shall view all orders across all users, update order status, and delete orders. Orders display the customer name via a JOIN.	Must Have	`OrderController`, `Order` model
FR-08	Order Detail Management (CRUD)	Admin shall manage individual line items within orders: product, quantity, and computed price. Price auto-calculates on product selection.	Must Have	`OrderDetailController`, `OrderDetail` model
FR-09	Role-Menu Mapping (CRUD)	Admin shall assign which menu categories are visible to which user types via the `usertype_menu` join table.	Should Have	`UserTypeMenuController`, `UserTypeMenu` model
FR-10	Browse Menu	Customers shall browse all available products grouped by menu category, with item names and prices displayed on cards.	Must Have	`customer/browse_menu.php` with product grouping logic
FR-11	Interactive Cart	Customers shall increment/decrement item quantities; the cart updates totals in real time without a page reload using JavaScript.	Must Have	JavaScript cart in `browse_menu.php`; cart modal
FR-12	Place Order	Customers shall submit their cart to create an order record and corresponding order detail records in a single transaction.	Must Have	`CustomerDashboardController::placeOrder()`; `Order::create()` returns `lastInsertId`
FR-13	My Orders	Customers shall view a history of all their own orders, filtered by their `user_id`, with status badges.	Must Have	`customer/my_orders.php`; array_filter on `user_id`
FR-14	Print Invoice	Any user shall print a formatted invoice for any of their orders. Admin can print invoices for all orders. The print layout hides navigation and action buttons.	Must Have	`app/views/invoice.php`; `@mediaprint` CSS
FR-15	Logout	The system shall destroy the PHP session and redirect the user to the login page when the logout link is clicked.	Must Have	`LoginController::logout()`; `session_destroy()`

4 Non-Functional Requirements

ID	Category	Description	Code Reference
NFR-01	Security	All user passwords are stored using the bcrypt algorithm via PHP's `PASSWORD_DEFAULT` constant (cost factor 10). PDO prepared statements with named placeholders are used for every database query, preventing SQL injection. Session-based access control blocks unauthenticated access to all protected routes.	`User::create()` calls `password_hash()`; `User::findByEmail()` calls `password_verify()`; all models use `PDO::prepare()`
NFR-02	Maintainability	The system follows the MVC architectural pattern: models handle data access, controllers handle HTTP logic, and views handle presentation. The `CrudInterface` enforces a uniform four-method contract across all seven models.	`CrudInterface.php` on all 7 models; one controller per entity
NFR-03	Usability	The interface uses a responsive CSS grid layout with no external CSS frameworks. CRUD operations open in modal dialogs, avoiding full-page reloads. Order statuses are colour-coded with badges (green = delivered, amber = pending, red = cancelled) for fast visual scanning.	`assets/css/style.css`; modal-overlay pattern in all CRUD views; JavaScript `openModal()` / `closeModal()` helpers
NFR-04	Reliability	All database operations use PDO with `ERRMODE_EXCEPTION`, so failures surface as exceptions rather than silent failures. The `Order::create()` method returns the new row's ID directly via `lastInsertId()` on the same connection, preventing race conditions.	`Database::connect()` sets `PDO::ATTR_ERRMODE`; `Order::create()` returns `$this->conn->lastInsertId()`
NFR-05	Portability	The system has zero Composer or npm runtime dependencies. It runs on any LAMP, WAMP, or XAMPP stack with PHP $\geq$ 7.4 and MySQL 5.7+. No `.htaccess` rewrite rules are required; the front controller is accessed directly via `public/index.php`.	No `composer.json`; pure PHP, no framework
NFR-06	Printability	The invoice page includes `@media print` CSS rules that hide the navigation bar, buttons, and all chrome, producing	`invoice.php`; `@media print` in `style.css`

5 Detailed Class Descriptions

The system is composed of **17 classes** across four layers: configuration, interface, models, and controllers. All model classes implement `CrudInterface`.

Architecture Note

Interface Realization: `CrudInterface` declares `create($data)`, `read()`, `update($id, $data)`, and `delete($id)`. Every model class implements this interface, ensuring a uniform CRUD contract enforced at compile time by PHP.

Configuration & Interface Layer

Class	File	Methods	Responsibility
`Database`	`config/database.php`	`connect()`	Creates and returns a PDO connection. Sets error mode to `ERRMODE_EXCEPTION`. Host, DB name, username, and password are private attributes.
`CrudInterface`	`interfaces/CrudInterface.php`	`create`, `read`, `update`, `delete`	PHP interface defining the four-method CRUD contract enforced on all seven model classes.

Model Layer (7 Models — all implement CrudInterface)

Class	File	Extra Methods	Responsibility
`User`	`models/User.php`	`findByEmail($e)`	Manages `users` table. `create()` hashes the password with `password_hash()`. `read()` JOINs `user_types` to include role name.
`UserType`	`models/UserType.php`	—	Manages `user_types` table. Stores role names such as *Admin* and *Customer*.
`Menu`	`models/Menu.php`	—	Manages `menu` table (food categories). Stores name and description.
`Product`	`models/product.php`	—	Manages `products` table. Each product has a name, price, and foreign key to `menu`. `read()` JOINs `menu`.
`Order`	`models/Order.php`	—	Manages `orders` table. `create()` returns `lastInsertId()` for use by `OrderDetail`. `read()` JOINs `users` for the customer name.
`OrderDetail`	`models/orderDetail.php`	—	Manages `order_details` line items. Stores order ID, product ID, quantity, and computed price. `read()` JOINs `products`.
`UserTypeMenu`	`models/UserTypeMenu.php`	—	Manages `usertype_menu` join table. Maps user type IDs to menu IDs. `read()` JOINs both `user_types` and `menu`.

Controller Layer (8 Controllers)

Class	Methods	Responsibility
`LoginController`	`showLogin()`, `login()`, `logout()`	Renders login form; verifies credentials with `password_verify()`; sets `$_SESSION` on success; calls `session_destroy()` on logout.
`UserController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for users. Passes user types to the view for the role dropdown.
`UserTypeController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for user types (roles).
`MenuController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for menu categories.
`ProductController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for products. Passes menu list for the category dropdown.
`OrderController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for orders. Passes user list for the customer dropdown.
`OrderDetailController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for order line items. Passes orders and products for dropdowns; price auto-computed client-side.
`UserTypeMenuController`	`index()`, `store()`, `update()`, `delete()`	Full HTTP CRUD handler for role-menu mappings.

6 Use Case Descriptions

Actors

Admin — System administrator with full access. **Customer** — Registered user with order access.

UC-01 Login

Actor(s)	Admin, Customer
Trigger	User navigates to `index.php?action=login`
Main Flow	1. System displays login form. 2. User enters email and password and submits. 3. `LoginController::login()` fetches user by email. 4. `password_verify()` checks the bcrypt hash. 5. On success: session variables set; Admin → admin dashboard, Customer → customer dashboard.
Alt. Flow	If credentials are invalid, redirect to login with `error=1` query param.
Outcome	Authenticated PHP session created; user on their role dashboard.

UC-02 Manage Users (Admin CRUD)

Actor(s)	Admin
Trigger	Admin clicks **Users** in the navigation bar
Main Flow	1. `UserController::index()` fetches all users (with role names). 2. Admin sees a table of users with Edit / Delete buttons. 3. **Create:** Click "+ Add User" → modal form → POST to `user_create`. 4. **Update:** Click "Edit" → modal pre-filled → POST to `user_update`. 5. **Delete:** Click "Delete" → confirm dialog → GET `user_delete&id=N`.
Outcome	User record created, updated, or removed in `users` table.

UC-03 Manage Menus, Products, User Types, Role-Menu Mapping

Actor(s)	Admin
Trigger	Admin clicks the relevant item in the navigation bar
Main Flow	1. Controller fetches the relevant records. 2. Admin views the table with inline Edit / Delete actions. 3. Create / Update / Delete follow the same modal pattern as UC-02.
Scope	Applies to: `menu`, `products`, `user_types`, `usertype_menu`
Outcome	Record persisted, updated, or deleted in the respective table.

UC-04 Manage Orders & Order Details

Actor(s)	Admin
Trigger	Admin clicks **Orders** or **Order Details**
Main Flow	1. Controller fetches records (with JOINs for customer/product names). 2. Admin can update order status (pending / processing / delivered / cancelled). 3. Admin can delete an order or individual line item. 4. Admin can click **Invoice** to open the printable invoice in a new tab.
Outcome	Order status updated; line items managed; invoice accessible.

UC-05 Browse Menu & Add to Cart

Actor(s)	Customer
Trigger	Customer clicks **Browse Menu**
Main Flow	1. Products are fetched and grouped by menu category. 2. Each product card shows name, price, and a quantity spinner. 3. Customer adjusts quantities; cart total updates in real-time (JavaScript). 4. Customer clicks **View Cart** to review items before ordering.
Outcome	Cart populated in browser memory, ready to be submitted as an order.

UC-06 Place Order

Actor(s)	Customer
Trigger	Customer clicks **Place Order** in the cart modal
Main Flow	1. Cart is POSTed to `index.php?action=place_order`. 2. Server computes total from product prices × quantities. 3. `Order::create()` inserts a row into `orders` and returns `lastInsertId()`. 4. `OrderDetail::create()` inserts one row per cart item into `order_details`. 5. Redirect to My Orders with a success message.
Outcome	New order with status *pending* created; order details persisted.

UC-07 View My Orders

Actor(s)	Customer
Trigger	Customer clicks **My Orders** in the navigation bar
Main Flow	1. All orders are fetched; filtered client-side by the session `user_id`. 2. Orders are displayed newest-first with colour-coded status badges. 3. Each row has a **Print Invoice** link.
Outcome	Customer views their complete order history.

UC-08 Print Invoice

Actor(s)	Admin, Customer
Trigger	User clicks **Invoice** button for an order
Main Flow	1. `CustomerDashboardController::printInvoice()` verifies that the requesting user owns the order (or is Admin). 2. Order details are fetched and rendered in `app/views/invoice.php`. 3. Invoice opens in a new tab. 4. User clicks **Print Invoice** to trigger the browser print dialog.
Outcome	Formatted paper invoice printed. Navigation hidden by `@mediaprint` CSS.

UC-09 Logout

Actor(s)	Admin, Customer
Trigger	User clicks **Logout** in the navigation bar
Main Flow	1. Request routed to `LoginController::logout()`. 2. `session_destroy()` terminates the session. 3. User redirected to the login page.
Outcome	Session destroyed; user must log in again to access the system.

7 Database Schema

The system uses **7 MySQL tables** with foreign-key relationships enforced at the database level.

#	Table	Key Columns	Description
1	`user_types`	`id`, `type_name`	Roles: Admin, Customer. Referenced by `users`.
2	`users`	`id`, `name`, `email`, `password`, `user_type_id`	System accounts. Password stored as bcrypt hash. FK to `user_types`.
3	`menu`	`id`, `name`, `description`	Food categories (Breakfast, Main Course, Desserts, Beverages).
4	`products`	`id`, `name`, `price`, `menu_id`	Individual food items. FK to `menu`.
5	`orders`	`id`, `user_id`, `total_price`, `status`, `created_at`	Customer orders. Status ENUM: pending / processing / delivered / cancelled. FK to `users`.
6	`order_details`	`id`, `order_id`, `product_id`, `quantity`, `price`	Line items per order. FK to both `orders` and `products`.
7	`usertype_menu`	`id`, `user_type_id`, `menu_id`	Maps roles to accessible menus. Unique constraint on `(user_type_id,menu_id)`. FK to `user_types` and `menu`.

Entity Relationship Summary

$$\texttt{user_types} \xleftarrow{1} \texttt{users} \xrightarrow{N} \texttt{orders} \xrightarrow{N} \texttt{order_details} \xrightarrow{N} \texttt{products} \xrightarrow{N} \texttt{menu}$$

$$\texttt{user_types} \xleftarrow{N} \texttt{usertype_menu} \xrightarrow{N} \texttt{menu}$$

Food Ordering System

Software Requirements Specification

Version 1.0 | March 23, 2026

Prepared for: Fundamentals of Software Engineering — Phase 1